Lecture 11: Text as Data

LSE ME314: Introduction to Data Science and Machine Learning (https://github.com/me314-lse)

2026-07-29

Ryan Hübert

Slides last updated: 28 July 2026

From yesterday to today

Yesterday: how the "real" world becomes digital data.

- ➜ Everything in a computer is bytes
- ➜ Audio, images and maps are digitised by *sampling*
- ➜ Once digitised, these data are table-like

Today: text.

- ➜ Text is *already* digital: no sampling involved
- ➜ But it is not table-like: text is stored in **strings**

Today's trip: from strings to tables

The challenge with text: making it **tabular**.

But first we need text to work with, and most of it lives on the web.

So, in two steps:

1. **Get it**: text from webpages and APIs
2. **Process it**: from raw text to a matrix

Getting data from the web

Data on the web

Where does text data come from?

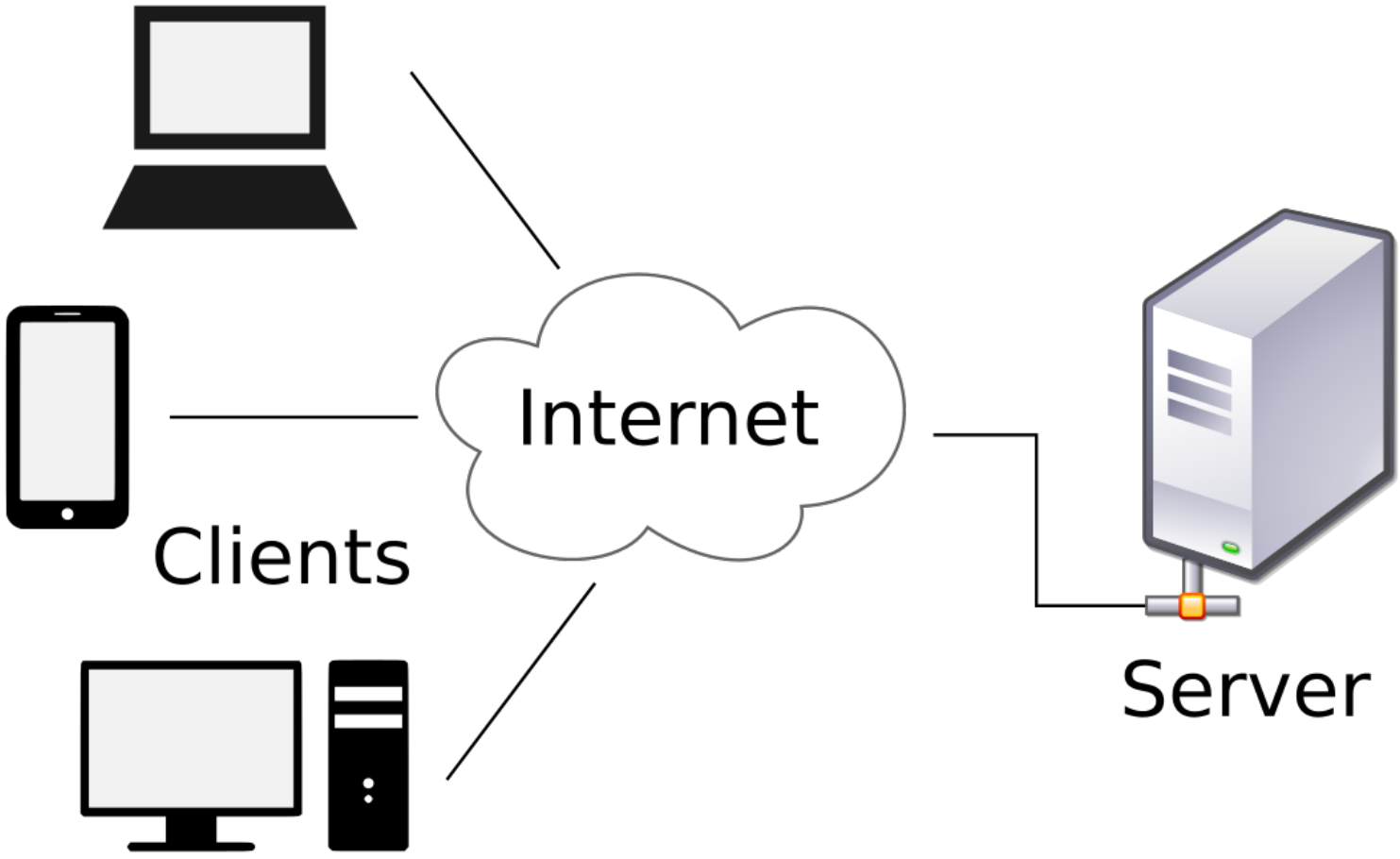
Very often: the web, where a lot of data is already digital.

- ➜ Archived datasets, administrative data, social media posts, speeches, news...
- ➜ Same theme as today: it isn't handed to you as a clean table

To get it, you do two things:

1. request the data from a server, and
2. parse the structure it arrives in.

The client–server model



The client–server model

Client: User computer, tablet, phone, software application, etc.

Server: Web server, mail server, file server, etc.

In **request–response systems**:

- ➜ Client makes **request** to the server
- ➜ Depending on what you want to get, the request might be
  - ➜ **HTTP**: Hypertext Transfer Protocol
  - ➜ **HTTPS**: Hypertext Transfer Protocol Secure
  - ➜ **SMTP**: Simple Mail Transfer Protocol
  - ➜ **FTP**: File Transfer Protocol
- ➜ Server returns **response**

Anatomy of a request

A client sends a request with several components:

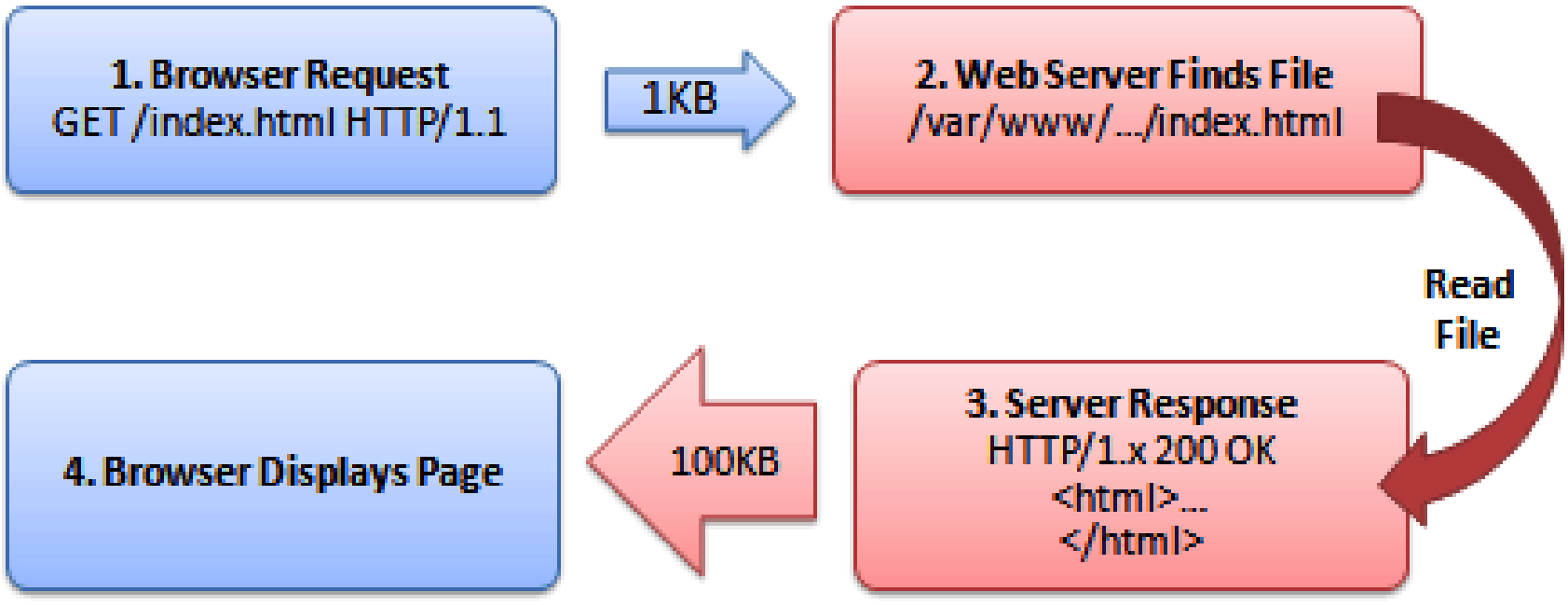
- **URL**: where the request is sent, a remote "path"
- **Method**: what kind of action you're asking for
  - `GET` to retrieve (digital) data
  - `POST` to send or submit data
  - `PUT` or `DELETE` to modify or remove data
- **Headers**: metadata about the request represented as key-value pairs, e.g.:
  - **User-Agent**: who/what is making request
  - **Authorization**: provides credentials like tokens
- **Body**: data being sent (only for `POST`/`PUT` requests)

Anatomy of a response

A server sends back a response with several components

- **Status code**: was request fulfilled? For example: `200 OK`, `403 Forbidden`, or `404 Not Found`
  - Status codes 4xx are client errors; 5xx are server errors
- **Headers**: info about the response, for example
  - **Content-Type**: what kind of data
  - **Content-Length**: length in bytes
  - **RateLimit-Remaining**: how many requests before throttling
- **Body**: the content requested, such as
  - HTML if you're accessing a webpage
  - JSON or XML if you're accessing an API

Example request and response



Source: StackOverflow

A simple GET request in R

Can use {httr} to make requests in R, e.g.:

```r
library("httr")
response <- GET(url = "https://github.com/me314-lse/")
print(response)
```

```
Response [https://github.com/me314-lse/]
  Date: 2026-07-26 10:56
  Status: 200
  Content-Type: text/html; charset=utf-8
  Size: 275 kB


<!DOCTYPE html>
<html
  lang="en"
  data-color-mode="auto" data-light-theme="light" data-dark-theme="dark"
  data-a11y-animated-images="system" data-a11y-link-underlines="true"
  >
...
```

This returned HTML, which is used for webpages.

What comes back: raw bytes

Content should be an HTML webpage but it's raw bytes:

```
response$content[1:20]   # first 20 bytes
```

```
[1] 0a 0a 3c 21 44 4f 43 54 59 50 45 20 68 74 6d 6c 3e 0a 3c 68
```

Fortunately, the response tells you how to read the bytes:

```
response$headers$`content-type`
```

```
[1] "text/html; charset=utf-8"
```

These bytes are UTF-8 encoded text, so we can do:

```
rawToChar(response$content[1:20])
```

```
[1] "\n\n<!DOCTYPE html>\n<h"
```

Can extract text directly: `content(response, "text")`

The structure of a webpage

When you request a webpage in a web browser, the server returns plain-text files that a browser "renders" into something nice.

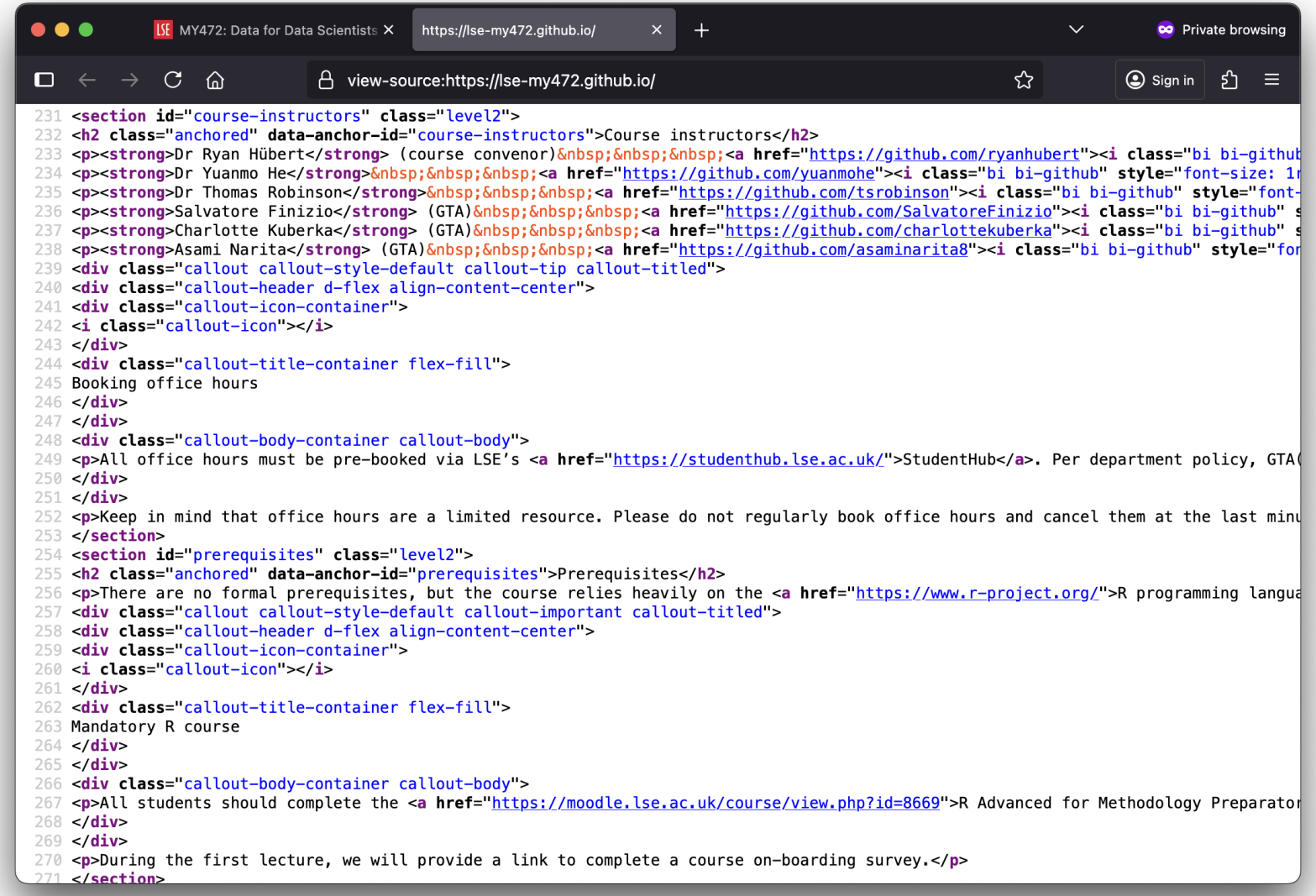
We call these files the **source code**. Some examples:

- **HTML** (Hypertext Markup Language): the structure and content of the page
- **CSS** (Cascading Style Sheets): the formatting
- **JavaScript**: dynamic behaviour, can change content *after* the page loads

You can view the source code in any browser (`View Source`, or *inspect element* in the developer tools).

When you make a "simple" request for a webpage via `{httr}`, you just get the webpage itself, in HTML.

Source code in the browser




```
231 <section id="course-instructors" class="level2">
232 <h2 class="anchored" data-anchor-id="course-instructors">Course instructors</h2>
233 <p><strong>Dr Ryan Hübert</strong> (course convenor)   <a href="https://github.com/ryanhubert"><i class="bi bi-github
234 <p><strong>Dr Yuanmo He</strong>   <a href="https://github.com/yuanmohe"><i class="bi bi-github" style="font-size: 1r
235 <p><strong>Dr Thomas Robinson</strong>   <a href="https://github.com/tsrobinson"><i class="bi bi-github" style="font-
236 <p><strong>Salvatore Finizio</strong> (GTA)   <a href="https://github.com/SalvatoreFinizio"><i class="bi bi-github" s
237 <p><strong>Charlotte Kuberka</strong> (GTA)   <a href="https://github.com/charlottekuberka"><i class="bi bi-github" s
238 <p><strong>Asami Narita</strong> (GTA)   <a href="https://github.com/asaminarita8"><i class="bi bi-github" style="for
239 <div class="callout callout-style-default callout-tip callout-titled">
240 <div class="callout-header d-flex align-content-center">
241 <div class="callout-icon-container">
242 <i class="callout-icon"></i>
243 </div>
244 <div class="callout-title-container flex-fill">
245 Booking office hours
246 </div>
247 </div>
248 <div class="callout-body-container callout-body">
249 <p>All office hours must be pre-booked via LSE's <a href="https://studenthub.lse.ac.uk/">StudentHub</a>. Per department policy, GTA(
250 </div>
251 </div>
252 <p>Keep in mind that office hours are a limited resource. Please do not regularly book office hours and cancel them at the last minu
253 </section>
254 <section id="prerequisites" class="level2">
255 <h2 class="anchored" data-anchor-id="prerequisites">Prerequisites</h2>
256 <p>There are no formal prerequisites, but the course relies heavily on the <a href="https://www.r-project.org/">R programming langua
257 <div class="callout callout-style-default callout-important callout-titled">
258 <div class="callout-header d-flex align-content-center">
259 <div class="callout-icon-container">
260 <i class="callout-icon"></i>
261 </div>
262 <div class="callout-title-container flex-fill">
263 Mandatory R course
264 </div>
265 </div>
266 <div class="callout-body-container callout-body">
267 <p>All students should complete the <a href="https://moodle.lse.ac.uk/course/view.php?id=8669">R Advanced for Methodology Preparator
268 </div>
269 </div>
270 <p>During the first lecture, we will provide a link to complete a course on-boarding survey.</p>
271 </section>
```

HTML is a tree of elements

An HTML document contains **elements**: the objects that make up a webpage (text, images, links, scripts).

Elements are written with **tags**, markup in angle brackets, arranged in a tree.

- ➜ most elements have an **opening tag** and a **closing tag**, e.g. `<p>Hello world!</p>`
- ➜ **void elements** are self-contained, with no closing tag: `<img>`, `<link>`, `<br>`

HTML is a tree of elements

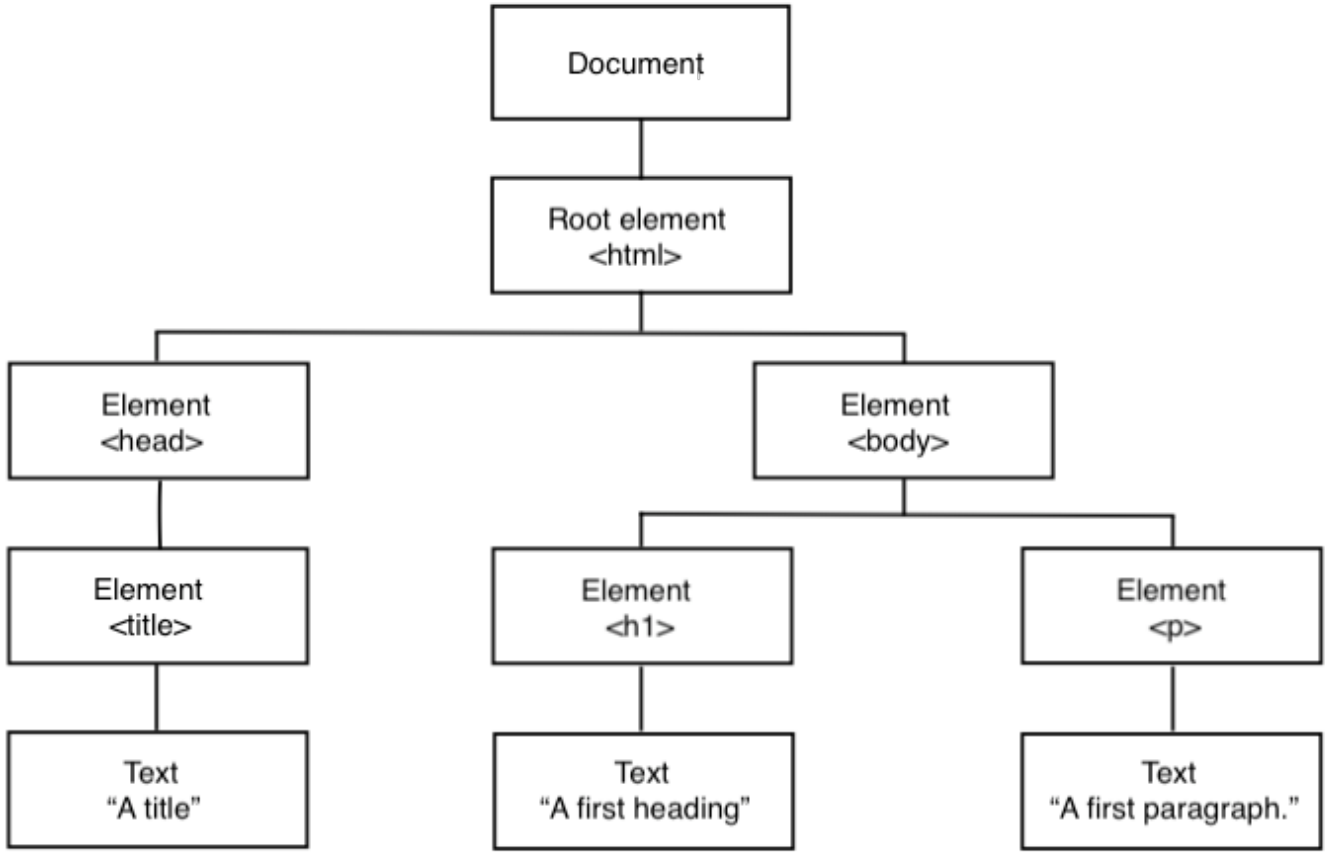
```
<!DOCTYPE html>
<html>
    <head><title>A title</title></head>
    <body>
        <h1>A first heading</h1>
        <p>A first paragraph.</p>
    </body>
</html>
```

Key vocabulary (like file paths):

- ➜ `<html>` is the **root element**
- ➜ `<head>` is the **parent element** of `<title>`
- ➜ `<h1>` is a **child element** of `<body>`
- ➜ `<head>` and `<p>` are **descendant elements** of `<html>`

`<!DOCTYPE html>` is metadata inside the file (indicating HTML).

HTML is a tree of elements



HTML attributes

Tags can carry **attributes**: extra detail about the element.

These are how we'll *find* the data we want, e.g.

- `<a href="https://www.google.com/">Link</a>`
- `<img src="site_logo.png">`
- `<div class="my_class">Some content.</div>`
- `<p id="firstParagraph">My first paragraph.</p>`

Note `class` and `id`: the labels we'll use to target elements.

Is the data actually in the source?

The crucial question before scraping: is the data in the *initial* HTML the server sends?

- ➜ **Static sites** send fixed HTML: straightforward to scrape
- ➜ **Dynamic sites** build the HTML on the fly (server- or JavaScript-side)
- ➜ **Single-page apps** send a minimal skeleton, then JavaScript injects the content, so the data may not be in the source *at all*

A simple `GET` gets you the *initial* source.

If the data isn't there, a plain request won't find it.

Identify yourself, and be polite

We can now request a page and read its HTML. Before we scrape: two key parts of ethical data collection from the web.

1. Identify yourself clearly with a **User-Agent** header:

```
ua <- "ME314-2026 (educational use; contact: USERNAME@lse.ac.uk)"
response <- GET(url = "https://lse.ac.uk/", user_agent(ua))
```

This tells the webserver's administrator who you are and how to contact you if your script causes issues. Use your email address!

2. Minimise your impact on the server: include a delay between requests.

```
Sys.sleep(0.5)   # in seconds; place delays after each request
```

Set reasonable delays, and use good judgement.

Scraping a webpage

Web scraping: extracting data from HTML built for human eyes.

The **rules of the game**:

1. Respect the hosting site's wishes
   - ➜ check if an API exists or if data is available for download
   - ➜ respect copyright and ethics: what are you allowed to do?
   - ➜ keep in mind where data comes from and give credit
   - ➜ some websites disallow scrapers via a `robots.txt` file
2. Limit your bandwidth use
   - ➜ wait some time after each request ("polite delays")
   - ➜ scrape only what you need, and just once
3. Read documentation
   - ➜ is there a batch download option? rate limits? can you share the data?

Parsing webpages

Targeting elements

To extract data, you "point at" elements using their tags and attributes (tag, `class`, `id`).

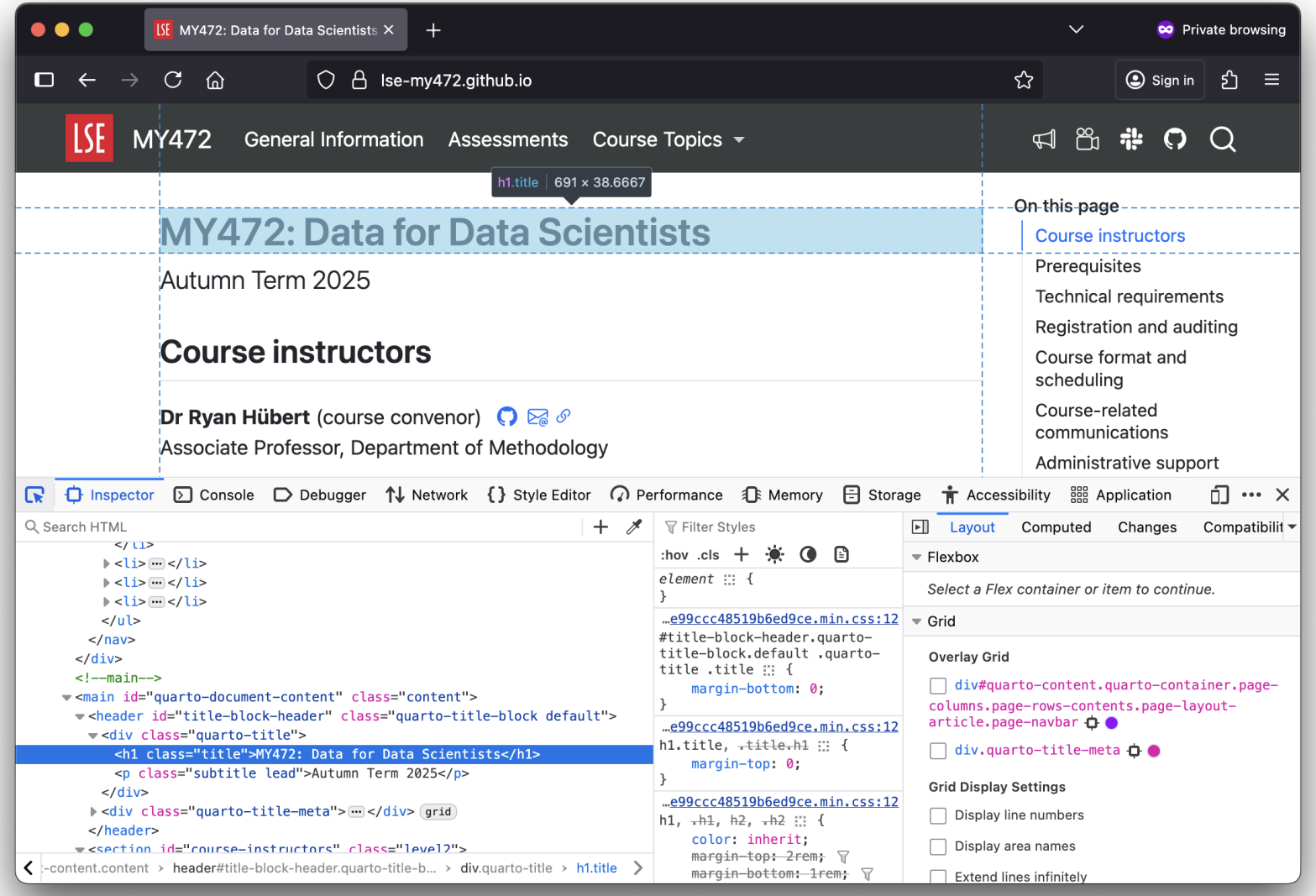
There are two main ways to do this:

- ➜ **CSS selectors**
- ➜ **XPath**

You rarely write these by hand: use *inspect element* in the browser to find the right label

- ➜ We'll practise CSS selectors hands-on in the seminar

Inspecting elements in the browser



Scraping a table with rvest

{rvest} requests *and* parses HTML. Let's scrape the most populous urban areas in the UK.

```
library("rvest")
library("tidyverse")

url <- paste0("https://en.wikipedia.org/wiki/",
              "List_of_urban_areas_in_the_United_Kingdom")
Sys.sleep(1)                                  # a polite delay!
html <- read_html(url)                        # GET + parse in one step

tables <- html_table(html, fill = TRUE)   # a LIST of tables
lapply(tables, dim)                           # inspect before choosing!
```

```
[[1]]
[1] 2 3
[[2]]
[1] 91  9
[[3]]
[1] 5 2
```

Don't blindly hardcode `tables[[2]]`: *inspect* the tables first.

Scraping a table with rvest

```r
data_pop <- tables[[2]]
colnames(data_pop)[2:3] <- c("city_name", "population")

data_pop <- data_pop |>
  select(city_name, population) |>
  mutate(
    city_name  = str_remove(city_name, "\\[.*\\]"),        # footnotes
    population = str_remove(population, "\\[.*"),          # footnotes
    population = as.numeric(str_remove_all(population, ","))
  ) |>
  arrange(desc(population))
head(data_pop)
```

```
# A tibble: 6 x 2
  city_name          population
  <chr>                   <dbl>
1 Greater London        9787426
2 Greater Manchester    2553379
3 West Midlands         2440986
4 West Yorkshire        1777934
5 Greater Glasgow        985290
6 Liverpool              864122
```

Scraping unstructured data with CSS selectors

Tables are easy, for everything else: target elements by tag and attribute.

Example: pull instructor names off a course page.

```
ins <- read_html("https://lse-my472.github.io") |>
    html_elements(css = "section#course-instructors")

# Get all `p` tags
ins <- html_elements(ins, css = "p")

# Only keep ones containing text "Dr" or "(GTA)"
ins <- ins[str_detect(html_text(ins), "(Dr |[(]GTA[)])")]

# The name sits in a <strong> tag
name <- ins |>
  html_elements(css = "strong") |>
  html_text()
print(name)
```

```
[1] "Dr Ryan Hübert"     "Dr Yuanmo He"       "Dr Thomas Robinson" "Sal
[5] "Charlotte Kuberka"  "Asami Narita"
```

When scraping is hard

Automated requests don't always work, e.g. many sites now *block* automated access:

```
GET("https://www.ssa.gov/oact/babynames/numberUSbirths.html",
    user_agent(ua))
```

```
Response [https://www.ssa.gov/oact/babynames/numberUSbirths.html]
  Date: 2026-07-28 14:31
  Status: 403
  Content-Type: text/html
  Size: 407 B
```

But also:

- ➜ The content may be delivered *dynamically*, so a plain `GET` won't return it
- ➜ Browser-automation tools (e.g. Selenium) can drive a real browser, but they're **out of scope** and increasingly fraught
- ➜ Never hide who you are to evade a server's policies

Webscraping ethics

Scraping has got harder: stricter terms of service, more aggressive blocking. A sensible checklist:

1. Is there an API or a direct download? Use it instead.
2. Does collecting this data seem sketchy (stealing work, exposing personal data, breaking laws)?
3. If you proceed: check `robots.txt` and the terms of service.
4. Identify yourself (User-Agent) and don't overwhelm the server (polite delays).
5. Got a 403? Re-evaluate step 2: sometimes it's technical, sometimes not.

Web APIs

Not every request is for a webpage

A request can ask a server for many different things; only *some* of them are webpages:

- ➜ A **webpage**: HTML, meant to be *rendered for a human* to read
- ➜ A **file**: a PDF, a CSV, an image
- ➜ **Data from an API**: usually JSON, *read by a program*

So there are two main routes to the same goal (a tidy dataset):

- ➜ **Scrape a webpage**: extract data from a document built for human eyes (what we just did)
- ➜ **Call an API**: ask for data that's already structured

Rule of thumb: if an API exists, use it. You've seen scraping is fiddly and fragile; an API is the clean route.

What is an API?

Application programming interface (API):

- ➜ defined set of functions, classes, or methods
- ➜ allows one piece of software to interact with another
- ➜ works without needing to understand its internal workings

APIs are not user interfaces, though they can power one

As an example: every R package is an API

A **web API** is the same idea across the internet:

- ➜ you send a structured request to a remote server
- ➜ it sends back data (not a webpage), most often as **JSON**

Endpoints, parameters and keys

To call a web API you build a URL from:

- ➜ an **endpoint**: the web location for the request
- ➜ **parameters**: how you customise what you're asking for
- ➜ endpoint + parameters = a customised request URL

Two realities of most APIs:

- ➜ **Keys / tokens**: many require you to register and pass a key (in the URL or a header)
- ➜ **Rate limits**: caps on calls per key/time; some APIs charge

Always read the terms of service, for the API *and* the data.

JSON: what an API sends back

APIs usually respond with **JSON** (JavaScript Object Notation): a lightweight format of **key-value pairs**.

```
{
  "USER261728": {
    "first_name": "John",
    "last_name": "Smith",
    "is_alive": true,
    "age": 27,
    "children": ["Catherine", "Thomas", "Trevor"],
    "spouse": null
  }
}
```

- ➜ Keys are strings; values can be strings, numbers, arrays, booleans, `null`
- ➜ Objects can be **nested**, mapping neatly onto an R `list`

JSON in R

In R, `fromJSON()` (from `jsonlite`) reads and parses JSON, rendering it as a `list` object:

```
library("jsonlite")
users <- fromJSON("simple_example.json")
users$USER261728$children
```

```
[1] "Catherine" "Thomas"    "Trevor"
```

Can pull info out using ordinary R `list` syntax.

Constructing an API call: UK Parliament

The UK Parliament's Members API returns data on MPs.

- ➜ It is public and needs no key.
- ➜ An **endpoint** from the API:
  `https://members-api.parliament.uk/api/Members/Search`
- ➜ Returns **JSON**

(Many APIs, especially commercial ones, do need a key: more on handling those shortly.)

Read the docs: https://developer.parliament.uk.

Build a request URL

Customise the request with **parameters**, added to the endpoint:

- ➜ use ? to start the parameters
- ➜ separate parameters with &
- ➜ replace spaces with %20

To search the Commons for MPs named "Abbott", the URL is:

```
https://members-api.parliament.uk/api/Members/Search?Name=Abbott&House=1
```

How did I know what parameters to use? I read the docs!

Making the call in R

You could paste this URL into a browser, but we want the data in R.

Build the URL yourself, then `GET` it:

```
library("httr")    # to make requests

endpoint <- "https://members-api.parliament.uk/api/Members/Search"

url <- endpoint |>
  paste0("?Name=Abbott") |>
  paste0("&House=1")

r <- GET(url)
r$status_code   # 200 is ok!
```

```
[1] 200
```

Parsing the JSON result

`content()` parses the JSON response into an R `list`:

```
result <- content(r)
result$items[[1]]$value$nameDisplayAs
```

```
[1] "Ms Diane Abbott"
```

```
result$items[[1]]$value$latestParty$name
```

```
[1] "Independent"
```

The JSON key-value structure becomes a nested R `list` you index with `$` and `[[ ]]`.

How did I know how to extract that info? Inspect + read the docs!

Never expose your keys

The Parliament API needed no key, but many APIs (especially commercial ones) do.

When they do, you're responsible for keeping the key secret.

Never hard-code a key or token in a script:

```
my_key <- "my key for everyone to see!"   # DON'T
```

- ➜ anyone who reads your code (or your GitHub repo) now has your key
- ➜ easy to expose by mistake, e.g. pushing to a public repository

A first step: keep keys out of code in an `.Renviron` file, read with `Sys.getenv()`. But an `.Renviron` file is still unencrypted plain text. Never push it to GitHub.

Store keys in your keychain

More secure: use your computer's **keychain**, operating-system software that stores secrets (like tokens) for use by other software.

- ➜ macOS: "Keychain Access"; Windows: "Credential Manager"
- ➜ not the same as a password manager, which is for human use

Access the keychain in R with `keyring`:

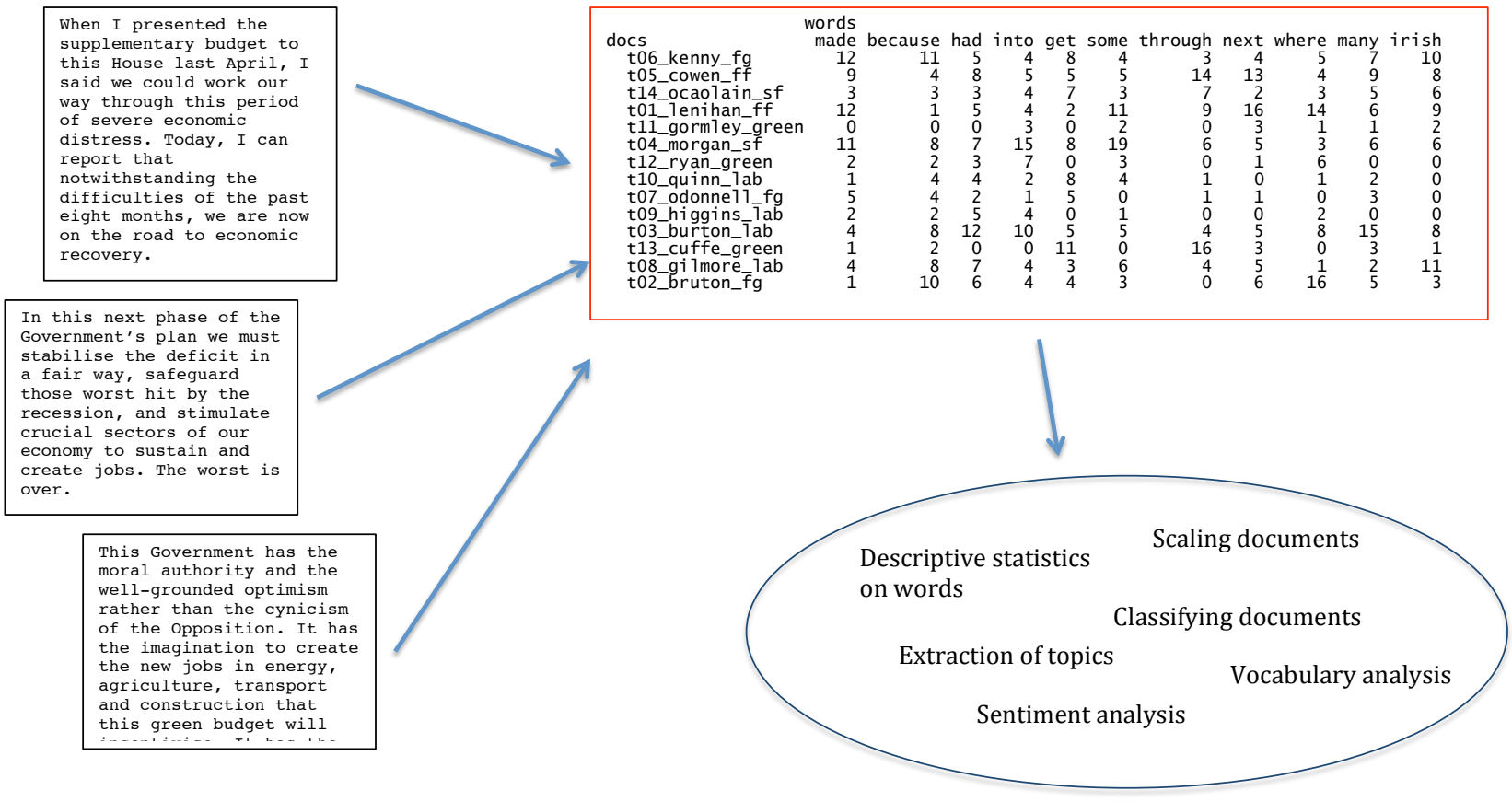
```
library("keyring")
key_set("some-api-key")              # add a key to the keychain
my_key <- key_get("some-api-key") # get it back, no plain text on disk
```

- ➜ the key never appears in your code or your repo

Processing text into structured data

Quantifying texts

We want to use quantitative (read: statistical) methods, so our goal is to conceptualise *texts* as *tabular data* (read: a matrix)



Quantifying texts

We'll be quantifying texts using the `quanteda` package in `R`

- ➜ `quanteda` was developed by a former LSE prof, Ken Benoit
- ➜ Offers a seamless package for text-related analysis

```
install.packages("quanteda")
install.packages("quanteda.textplots")
install.packages("quanteda.textstats")
```

There are other tools in Python, probably more common in industry

Document selection

A **document** is the fundamental unit of analysis for quantitative text analysis, for example:

- n-word sequences
- Sentences
- Pages
- Paragraphs
- Natural units (a speech, a poem, a manifesto)
- Aggregation of units (e.g. all speeches by party and year)

A collection of documents is called a **corpus**

How you define documents depends on your goal

Side note on terminology

Let's keep some words straight for this lecture:

- a **document** is the chosen unit of analysis; could be a full "document" in the colloquial sense, or not
- a **text** (countable noun) is what we often call a "document" colloquially, e.g. a novel, a speech, a legal opinion, a tweet
- **text** (uncountable noun) is a general word for a type of data, similar to "string"

I'll try to be consistent, but context will usually be informative

Example: US President Trump's tweets

To demonstrate quantifying texts, we'll use a corpus of US President Trump's tweets

- ➜ Corpus covers 2017 to mid-2018
- ➜ Available on the course GitHub repo

Data is stored in a `.json` file based on the Twitter API

- ➜ Convenient `{streamR}` package to load it

Example: US President Trump's tweets

Source: https://x.com/realDonaldTrump/status/815422340540547073

Example: US President Trump's tweets


```
{ "created_at": ["Sun Jan 01 05:00:10 +0000 2017"],
  "id": [8.15422340540547e+17],
  "id_str": ["815422340540547073"],
  "full_text": ["TO ALL AMERICANS-\n#HappyNewYear &amp; many blessings to
                 you all! Looking forward to a wonderful &amp; prosperous
                 2017 as we work together to #MAGA\U0001F1FA\U0001F1F8
                 https://t.co/UaBFaoDYHe"],
  "truncated": [false],
  "display_text_range": [[0],[148]],
  "entities": {"hashtags": [{"text":["HappyNewYear"],"indices":[[18],[31]]},
                            {"text":["MAGA"], "indices":[[141],[146]]}],
              "symbols": [],
              "user_mentions":[],
              "urls":[],
              "media": [{"id":[8.15422333510816e+17],
                         "id_str":["815422333510815746"],
                         "indices":[[149],[172]],
                         "media_url":["http://pbs.twimg.com/media/C1D2SsLVEAIn
                         "media_url_https":["https://pbs.twimg.com/media/C1D2S
                         "url":["https://t.co/UaBFaoDYHe"],
                         ...
```

Defining document features

Documents contain **features**, which can be:

- characters
- words
- word "stems" or "lemmas" (more later)
- word segments, especially for languages using compound words, such as German, e.g. *Saunauntensitzer*
- "word" sequences, especially when inter-word delimiters (usually white space) are not commonly used, e.g. in Chinese
- linguistic features, such as parts of speech
- coded or annotated text segments
- word embeddings (more later)

We assume texts can be represented by quantifying their features

The most common approach: bag of words

Most common approach: the **bag of words** model

- ➜ Single *words* are the relevant *features* of each document
- ➜ Documents are quantified by counting occurrences of words
- ➜ Word order does not matter
- ➜ Discards grammar and syntax

Why bag of words?

Most obvious reason: it's very simple

But also, context is often uninformative once you condition on the *presence* of words

- ➜ Individual word usage tends to be associated with a particular degree of affect, position, etc. without regard to context
- ➜ So presence of words by itself captures info about context

Plus, *single* words tend to be the most informative since co-occurrences of multiple words ("n-grams") are relatively rare

For our purposes: in social science applications bag of words works well most of the time (i.e. it's been validated *a lot*)

Why bag of words?

There are times where word order is important, e.g.

- **Text reuse**: plagiarism detection software used for social science applications, such as Corley (2007) and Grimmer (2010)
- **Parts of speech tagging**: tagging words with grammatical information, with applications such as Bamman and Smith (2014) and Handler et al (2016)
- **Named entity recognition (NER)**: finding and tagging words that are people, organisations, or places, with applications such as Copus, Hübert and Pellaton (2024)

Whether word order matters depends on the task

Implementing bag of words

Goal: get from a set of texts to a quantitative dataset

There is a basic four step process for implementing bag of words:

1. Choose unit of analysis
2. Tokenise
3. Reduce complexity
4. Create **document-feature matrix**

People often refer to this as **preprocessing**

The "reducing complexity" part is where most of the discretion is

You should justify the preprocessing steps you take

Tokenising

You start by **tokenising** each document:

- ➜ Split into an array of words, each called a **token**
- ➜ For English (and many other languages), use white space; trickier for logographic languages (e.g. Chinese)
- ➜ Tokens are *mostly* "words", but not always

A list of every distinct token in a corpus is a **vocabulary**

- ➜ Each element of the vocabulary is a **type**

Tokenising Trump's tweet

Original (plain) text of Trump tweet:

```
TO ALL AMERICANS-\n#HappyNewYear &amp; many blessings
to you all! Looking forward to a wonderful &amp;
prosperous 2017 as we work together to
#MAGA\U0001F1FA\U0001F1F8 https://t.co/UaBFaoDYHe
```

Tokenising Trump's tweet

Use white space to tokenise, with the {stringr} package:

```
library("stringr")
str_split(tweets$text[1], "\\s+")[[1]]
```

```
c("TO", "ALL", "AMERICANS-", "#HappyNewYear", "&amp;",
  "many", "blessings", "to", "you", "all!", "Looking",
  "forward", "to", "a", "wonderful", "&amp;",
  "prosperous", "2017", "as", "we", "work", "together",
  "to", "#MAGA", "https://t.co/UaBFaoDYHe")
```

Notice some issues here:

- ➜ some useless punctuation: "AMERICANS-"
- ➜ some "non-words" included: links, ampersands
- ➜ the words "to" and "TO" are considered different words

This will create a vocabulary that is too large and redundant

Reduce complexity: cleaning up formatting

To deal with these problems, we can remove "non-words" and punctuation, and make text lowercase

```
c("to", "all", "americans", "#happynewyear", "many",
  "blessings", "to", "you", "all", "looking", "forward",
  "to", "a", "wonderful", "prosperous", "as", "we",
  "work", "together", "to", "#maga")
```

Note:

- ➜ Had to manually get rid of symbols written in HTML syntax (e.g. `&`) using the pattern `"[&][#]?[A-z]+;"`
  - ➜ This is a **regular expression** ("regex"): flexible pattern matching for strings, taught in the seminar!
- ➜ Decided to keep Twitter hashtags intact (why?)

Reduce complexity: removing stop words

Stop words are words that occur very frequently in a language but do not provide much information

Some very common English stop words are:

```
a, able, about, across, after, all, almost, also, am, among, an, and,
any, are, as, at, be, because, been, but, by, can, cannot, could,
dear, did, do, does, either, else, ever, every, for, from, get, got,
had, has, have, he, her, hers, him, his, how, however, i, if, in,
into, is, it, its, just, least, let, like, likely, may, me, might,
most, must, my, neither, no, nor, not, of, off, often, on, only, or,
other, our, own, rather, said, say, says, she, should, since, so,
some, than, that, the, their, them, then, there, these, they, this,
tis, to, too, twas, us, wants, was, we, were, what, when, where,
which, while, who, whom, why, will, with, would, yet, you, your
```

For analysis, it's common to remove stop words

- ➜ But this depends on your task, see Pennebaker (2011)
- ➜ There are different lists out there, some longer, some shorter

Removing stop words from Trump's tweet

We can remove all the stop words (defined by the list above) from the Trump tweet:

```
c("americans", "#happynewyear", "many", "blessings",
  "looking", "forward", "wonderful", "prosperous",
  "work", "together", "#maga")
```

Reduce complexity: stemming and lemmatisation

Many words have multiple "forms" with different spellings, e.g.

- ➜ Tenses: "I see" and "I saw"
- ➜ Pluralisation: "family" and "families"
- ➜ Contractions: "cannot" and "can't"

So far, we have treated all these as different words

But you may wish to create **equivalence classes** of words considered to convey the same meaning

- ➜ Basic idea: for every token, identify the "root" word, and replace the token with the root word
- ➜ This reduces vocabulary, and can be quite useful for comparing across documents

You may or may not want to do this depending on your task

Reduce complexity: stemming and lemmatisation

Two common approaches:

1. **Lemmatisation**: the algorithmic process of converting words to their **lemma**

   ➜ A word's **lemma** is its "canonical form"

2. **Stemming**: removing the ends of words using a set of rules

Basic difference: stemmers operate on single words without knowledge of the context, lemmatisers are more powerful

➜ There is a computational tradeoff

Example: `producer`, `produce`, `produces`, `produced` all replaced with `produc`

Tools for each available in `quanteda`

Reduce complexity: stemming and lemmatisation

Lots of different ways to stem and lemmatise

Porter stemmer is most common, but gets many stems wrong:

- ➜ `policy` and `police` considered (wrongly) equivalent
- ➜ `general` becomes `gener`, `iteration` becomes `iter`

There are other corpus-based, statistical, and mixed approaches designed to overcome these limitations

Plus, sometimes stemming isn't appropriate: Schofield and Mimno (2016) find that "stemmers produce no meaningful improvement in likelihood and coherence (of topic models) and in fact can degrade topic stability"

Take away: you have to read and validate!

Reduce complexity: stemming and lemmatisation

We can use quanteda's default stemming function `tokens_wordstem()` on the Trump tweet, yielding:

```
c("american", "#happynewyear", "mani", "bless", "look",
  "forward", "wonder", "prosper", "work", "togeth",
  "#maga")
```

Note:

- quanteda uses the Snowball stemmer by default
- The stemmer did many transformations, e.g.:
  - americans → american
  - many → mani
  - blessing → bless
  - together → togeth

The document-feature matrix (DFM)

So far, we've just "pre-processed" one example document

You repeat this process for every document

- ➜ This yields a vocabulary of types for the corpus
- ➜ These are the *features* to be analysed (again: remember we're assuming bag of words!)

Each document uses some of the features in the vocabulary

- ➜ You can count how many times

A **document-feature matrix** (math: $\mathbf{W}$) is a matrix of N documents (rows) by J features (columns) where:

- ➜ each W_{ij} counts the number of times the jth feature appears in the ith document

The document-feature matrix (DFM)

All of Trump's tweets from January 2017:

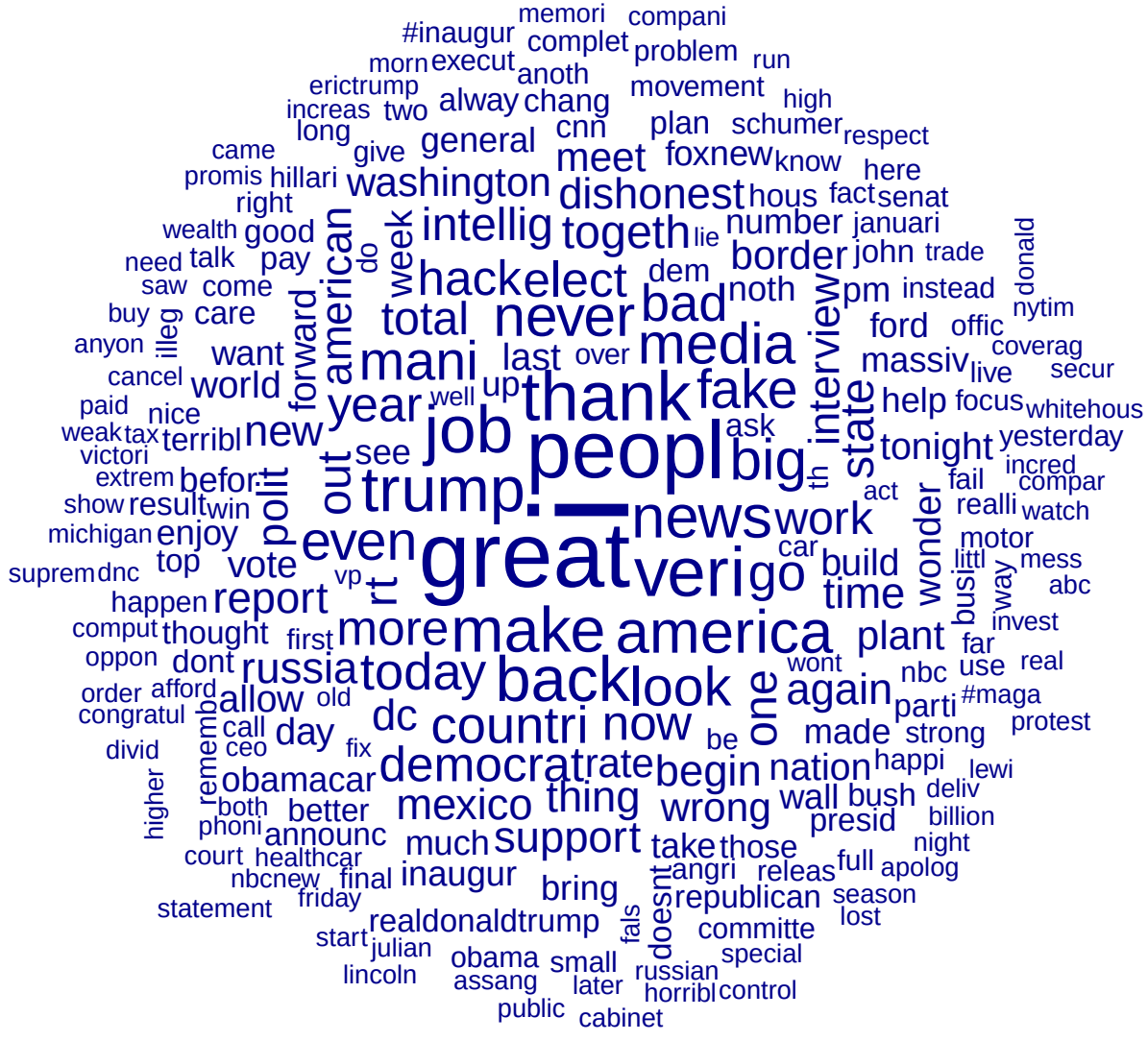
```
Document-feature matrix of: 212 documents, 1,039 features (98.94%
  sparse) and 0 docvars.
                     features
docs                  american #happynewyear mani bless look
  2017-01-01 05:00:10        1             1    1     1    1
  2017-01-01 05:39:13        0             1    0     0    0
  2017-01-01 05:43:23        0             0    0     1    1
  2017-01-01 05:44:17        0             0    0     0    0
  2017-01-01 06:49:33        0             0    0     0    0
[ reached max_ndoc ... 207 more documents, reached max_nfeat ...
  1,034 more features ]
```

This DFM has $J = 1,039$ features and $N = 212$ documents

➜ Each cell is a count, e.g. $W_{11} = 1$ and $W_{21} = 0$

Wordclouds

Wordclouds are basic visualisations of the data in a DFM



Not all tokens *should* be unigrams

When we tokenise using white space, we create **unigrams**

- ➜ (Technically: after we eliminated "non-words")

We could instead define tokens using a **collocation**, such as:

- ➜ **bigrams**: pairs of adjacent words
- ➜ **trigrams**: triples of adjacent words
- ➜ more generally: n-**grams**: n adjacent words

Example: `capital gains tax` can be represented as

- ➜ unigrams: `c("capital", "gains", "tax")`
- ➜ bigrams: `c("capital gains", "gains tax")`
- ➜ trigrams: `c("capital gains tax")`

Not all tokens *should* be unigrams

Why would you want to depart from unigrams?

1. You might want to do your entire analysis using n-grams instead of unigrams

   ➜ We won't do it here
   ➜ But it could be useful for situations where word order matters like simple text completion

2. Many collocations have independent meaning that you might want to retain in your analysis

   ➜ For example, `United Kingdom` makes more sense left as a bigram than as two unigrams
   ➜ In cases like this: need to manually define which phrases to leave as n-grams

How do you decide which words to collocate into n-grams?

Important collocations

$C(w^1\ w^2)$	w^1	w^2
80871	of	the
58841	in	the
26430	to	the
21842	on	the
21839	for	the
18568	and	the
16121	that	the
15630	at	the
15494	to	be
13899	in	a
13689	of	a
13361	by	the
13183	with	the
12622	from	the
11428	New	York
10007	he	said
9775	as	a
9231	is	a
8753	has	been
8573	for	a

Table 5.1 Finding Collocations: Raw Frequency. $C(\cdot)$ is the frequency of something in the corpus.

Source: Manning and Schütze (ch. 5)

Important collocations

$C(w^1\ w^2)$	w^1	w^2
80871	of	the
58841	in	the
26430	to	the
21842	on	the
21839	for	the
18568	and	the
16121	that	the
15630	at	the
15494	to	be
13899	in	a
13689	of	a
13361	by	the
13183	with	the
12622	from	the
11428	New	York
10007	he	said
9775	as	a
9231	is	a
8753	has	been
8573	for	a

Table 5.1 Finding Collocations: Raw Frequency. $C(\cdot)$ is the frequency of something in the corpus.

Source: Manning and Schütze (ch. 5)

Identifying collocations in quanteda

Find frequent collocations with `textstat_collocations()`

Let's find the common bigrams in the Trump tweet corpus

```
# A tibble: 5,335 x 6
   collocation   count count_nested length lambda     z
   <chr>         <int>        <int>  <dbl>  <dbl> <dbl>
 1 fake news       217            0      2   8.01  40.1
 2 tax cut         130            0      2   7.11  35.8
 3 make america     99            0      2   5.60  35.6
 4 unit state      101            0      2   7.25  30.6
 5 north korea     116            0      2   9.32  30.0
 6 news media       63            0      2   5.02  28.7
 7 america great    91            0      2   4.00  28.5
 8 great again      98            0      2   4.75  28.2
 9 illeg immigr     39            0      2   6.56  26.1
10 work hard        46            0      2   5.42  25.9
# i 5,325 more rows
```

Identifying collocations in quanteda

A subjective (but reasonable) judgement for most projects with this Trump tweet data:

➜ keep `United States` as a bigram

For example:

```
Having a great time hosting Prime Minister Shinzo Abe in the
United States! https://t.co/Fvjsac89qS https://t.co/OupKmRRuTI
https://t.co/smGrnWakWQ
```

Modified to:

```
Having a great time hosting Prime Minister Shinzo Abe in the
United_States! https://t.co/Fvjsac89qS https://t.co/OupKmRRuTI
https://t.co/smGrnWakWQ
```

Weighting with tf-idf

Can *weight* counts in cells of a DFM using **term frequency inverse document frequency (tf-idf) weighting**

- More weight to word counts with: high term frequency in document AND low document frequency in the whole corpus
- Weights tend to "filter out" common terms

Many variations for how to calculate, but basic idea is that the count in each cell ij of the DFM is replaced by:

$$\text{tfidf}_{ij} = \text{tf}_{ij} \times \text{idf}_j$$

The "simplest" version uses:

$$\text{tf}_{ij} = W_{ij} \qquad \text{idf}_j = \frac{N}{\text{df}_j}$$

where df_j is the number of documents containing feature j

Weighting with tf-idf

```
Document-feature matrix of: 212 documents, 1,039 features (98.94%
  sparse) and 0 docvars.
                     features
docs                  american #happynewyear     mani    bless
  2017-01-01 05:00:10 1.423246      2.025306 1.284943 2.025306
  2017-01-01 05:39:13 0             2.025306 0        0
  2017-01-01 05:43:23 0             0        0        2.025306
  2017-01-01 05:44:17 0             0        0        0
  2017-01-01 06:49:33 0             0        0        0
[ reached max_ndoc ... 207 more documents, reached max_nfeat ...
  1,035 more features ]
```

Sentiment analysis

A **dictionary** is a pre-defined list of features (e.g., words) associated with specific meanings

- ➜ It creates equivalence classes of features
- ➜ Frequently involves stemming/lemmatisation: transformation of all word forms to their "dictionary look-up form"

Two important components of dictionaries:

1. **Keys**: the labels for the equivalence class for the concept or canonical term
2. **Values**: (multiple) terms or patterns that are declared equivalent occurrences of the key class

Sentiment analysis

Most famous way to use a dictionary: **sentiment analysis**

- ➜ Quantify how "positive" or "negative" a document is based on word use
- ➜ The dictionary lists the "positive" and the "negative" words

Someone has to create the dictionary

A famous and widely used dictionary for sentiment analysis in political texts is **LexiCoder Sentiment Dictionary** (Young and Soroka 2012)

- ➜ It is validated with human-coded news content

Sentiment analysis

Included in quanteda as data_dictionary_LSD2015:

```
Dictionary object with 4 key entries.
- [negative]:
  - a lie, abandon*, abas* [ ... and 2,855 more ]
- [positive]:
  - ability*, abound*, absolv* [ ... and 1,706 more ]
- [neg_positive]:
  - best not, better not, no damag* [ ... and 1,718 more ]
- [neg_negative]:
  - not a lie, not abandon*, not abas* [ ... and 2,857 more ]
```

Source: https://www.snsoroka.com/data-lexicoder/

Sentiment analysis

Score a corpus by counting dictionary matches per document with `dfm_lookup()`:

```
Document-feature matrix of: 6 documents, 4 features (75.00%
  sparse) and 0 docvars.
                     features
docs                  negative positive neg_positive neg_negative
  2017-01-01 05:00:10        0        3            0            0
  2017-01-01 05:39:13        0        1            0            0
  2017-01-01 05:43:23        0        3            0            0
  2017-01-01 05:44:17        0        2            0            0
  2017-01-01 06:49:33        0        1            0            0
[ reached max_ndoc ... 1 more document ]
```

- ➜ Each document gets a count of positive and negative terms
- ➜ A simple sentiment score is the difference (or a ratio) of those counts
- ➜ As always: validate against what the documents actually say

Sentiment analysis

Kramer et al. (2014): study of 689,003 Facebook users

Experimentally manipulated content shown on news feeds to test the **emotional contagion hypothesis**

- ➜ Treatment 1: Positive content more visible on news feed
- ➜ Treatment 2: Negative content more visible on news feed
- ➜ Control: No news feed intervention

Use a dictionary method to measure sentiment of users' Facebook posts post-treatment, showing:

- ➜ positive content boosts positive posting, reduces negative
- ➜ negative content boosts negative posting, reduces positive

Huge concerns about ethics of the study; very controversial

Source: https://www.pnas.org/doi/10.1073/pnas.1320040111

Classification

You already saw classification tasks in the core ML lectures

Classification is a very common task for text

- ➜ Predicting authorship of documents
- ➜ Predicting demographic characteristics based on names
- ➜ Predicting partisan leanings based on speech patterns

You could imagine some neat "hybrid" approaches

- ➜ Voice recognition + speech patterns = predict who is speaking in a recorded audio file
- ➜ Image analysis (OCR) of protest signs + text analysis = categorise messages of a protest

Predicting sex at birth from a name

A worked example: predict the sex assigned at birth (SAAB) from a first name.

- ➜ Here: a **document** is a name, and a **feature** is a character.
- ➜ DFM: rows are names, columns are character sequences.
- ➜ Labelled data: ONS baby names, England and Wales, 2023
- ➜ Supervised task: train on labelled names, predict unseen ones

Data: https://www.ons.gov.uk/file?uri=/peoplepopulationandcommunity/birthsdeathsandmarriages/livebirths/datasets/babynamesenglandandwalesbabynamesstatistics

Build a character document-feature matrix

Tokenise names into characters, then into character n-grams:

```
df <- read_csv("uk_birth_names_2023.csv")

cnames <- corpus(df, text_field = "Name")
docvars(cnames, "SAAB") <- df$SAAB

characters <- tokens(cnames, what = "character")
characters <- tokens_ngrams(characters, n = 1:3)   # unigrams to trigrams

namesdfm <- dfm(characters)
namesdfm <- dfm_trim(namesdfm, min_docfreq = 20)   # drop very rare features
```

```
Document-feature matrix of: 13,813 documents, 1,111 features
  (98.79% sparse) and 4 docvars.
       features
docs    g r a c i e l g_r r_a a_c
  text1 1 1 2 1 1 1 2   1   1   1
  text2 0 0 2 0 1 0 0   0   0   0
  text3 0 0 2 0 1 0 1   0   0   0
  text4 0 0 0 0 0 0 0   0   0   0
  text5 0 0 1 0 0 1 0   0   0   0
  text6 1 1 1 1 1 3 1   1   1   1
[ reached max_ndoc ... 13,807 more documents, reached max_nfeat
  ... 1,101 more features ]
```

Split into training and test sets

Train the model on one part, evaluate perfomance on test set:

```
set.seed(2710)
smp <- sample(c("train", "test"), size = nrow(namesdfm),
              prob = c(0.80, 0.20), replace = TRUE)
train <- which(smp == "train")
test <- which(smp == "test")
```

An 80/20 split: 80% to train, 20% held out to measure performance.

Regularised regression: ridge

{glmnet} fits regularised regressions with cross-validation built in.

- ➜ `alpha = 0` is a ridge penalty
- ➜ `alpha = 1` is a lasso penalty
- ➜ Use 10-fold CV to pick the penalty λ

```
library("glmnet")
## Train model
ridge <- cv.glmnet(x = namesdfm[train, ],
                   y = docvars(cnames, "SAAB")[train],
                   alpha = 0, nfolds = 10, # Ridge with 10-fold CV on lambda
                   family = "binomial")    # Logistic regression
ridge_class <- predict(ridge, newx = namesdfm, type = "class")
print(as.character(ridge_class)[1:10])
```

```
[1] "girl" "girl" "girl" "girl" "girl" "girl" "girl" "girl" "girl" "boy"
```

Recall the ridge/lasso penalties from the core ML lectures.

Evaluate on the test set

```
cm <- table(df$SAAB[test], ridge_class[test])
cm                              # true class in rows, predicted in columns
sum(diag(cm)) / sum(cm)         # accuracy
cm[1,1] / sum(cm[,1])           # precision for boy class
cm[1,1] / sum(cm[1,])           # recall for boy class
```

```
       boy girl
  boy  1032  246
  girl  294 1196
[1] 0.8049133
[1] 0.7782805
[1] 0.8075117
```

- ➜ The **confusion matrix** shows correct and incorrect predictions
- ➜ Can evaluate performance with **precision**, **recall** and **accuracy**

Use it for prediction

The model does pretty well.

Let's use it to make a prediction about two names not in the data:

```
new_names <- c("Rocio", "Barack")

## Same pipeline as the training names, then align features
new_toks <- tokens(new_names, what = "character")
new_toks <- tokens_ngrams(new_toks, n = 1:3)
new_dfm  <- dfm(new_toks)
new_dfm  <- dfm_match(new_dfm, featnames(namesdfm))
docnames(new_dfm) <- new_names

predict(ridge, newx = new_dfm, type = "class")     # girl or boy
predict(ridge, newx = new_dfm, type = "response")  # Pr(girl)
```

```
       lambda.1se
Rocio  "girl"
Barack "boy"
       lambda.1se
Rocio   0.5514760
Barack  0.3057561
```

Regularised regression: lasso

Does lasso do better?

```
lasso <- cv.glmnet(x = namesdfm[train, ],
                   y = docvars(cnames, "SAAB")[train],
                   alpha = 1, nfolds = 10, family = "binomial")

lasso_class <- as.character(predict(lasso, newx = namesdfm, type = "class"))

cm <- table(df$SAAB[test], lasso_class[test])
cm                                  # true class in rows, predicted in columns
sum(diag(cm)) / sum(cm)             # accuracy
cm[1,1] / sum(cm[,1])               # precision for boy class
cm[1,1] / sum(cm[1,])               # recall for boy class
```

```
       boy girl
  boy  1055  223
  girl  286 1204
[1] 0.8161127
[1] 0.7867263
[1] 0.8255086
```

Lasso does a tad better, but is comparable.

➜ Pop quiz: should I now choose lasso as my main model?

How LLMs tokenise: byte pair encoding

Modern language models (Lecture 12) tokenise differently.

You already saw: texts must be broken up into countable features

- ➜ We call this **tokenisation**
- ➜ The resulting features are **tokens**

LLMs use **modified byte pair encoding (BPE)** tokenisers

- ➜ Tokens are bundles of commonly occurring consecutive substrings of Unicode characters
- ➜ A lookup table (like a character set) where every distinct token has an ID number
- ➜ Keep in mind: different LLMs may use different tokenisers

E.g., OpenAI's recent models use the `o200k_base` tokeniser

- ➜ It has about 200,000 unique tokens

How LLMs tokenise: byte pair encoding

OpenAI's tokeniser is available in R via the `rtiktoken` package

```
library("rtiktoken")
get_tokens(text = "London School of Economics",
           model = "gpt-4o")
```

```
[1] 51162  6586   328 56779
```

```
get_token_count(text = "London School of Economics",
                model = "gpt-4o")
```

```
[1] 4
```

How LLMs tokenise: byte pair encoding

What makes BPE tokenisers different from "traditional" tokenisers:

```
get_tokens(text = "london school of economics",
           model = "gpt-4o")
```

```
[1]    75  8552  3474   328 48229
```

The tokeniser is case sensitive (so are "traditional" tokenisers, before lowercasing)

But with this BPE tokeniser, "london" is two tokens!

- ➜ Why? Short answer: "London" is a more common set of co-occurring bytes than "london"
- ➜ BPE tokenisers yield more efficient representations

Play around here: https://platform.openai.com/tokenizer

Word embeddings

Bag of words: each word has a distinct, unique meaning

➜ In math terms: we treat words as **one-hot encodings**

Concrete example: consider a vocabulary of three words:
`c("cat", "dog", "rat")`

Can represent each word in **vector format**:

- ➜ The word `cat` is `(1,0,0)`
- ➜ The word `dog` is `(0,1,0)`
- ➜ The word `rat` is `(0,0,1)`

(Should be obvious why this is called "one-hot encoding")

These are *orthogonal*: zero similarity between the words

Word embeddings

But what if words are actually representations of a smaller group of concepts, where each word is a *mix* of concepts?

E.g. what if these words can be represented in a two dimensional **embedding** (e.g. two distinct "concepts")

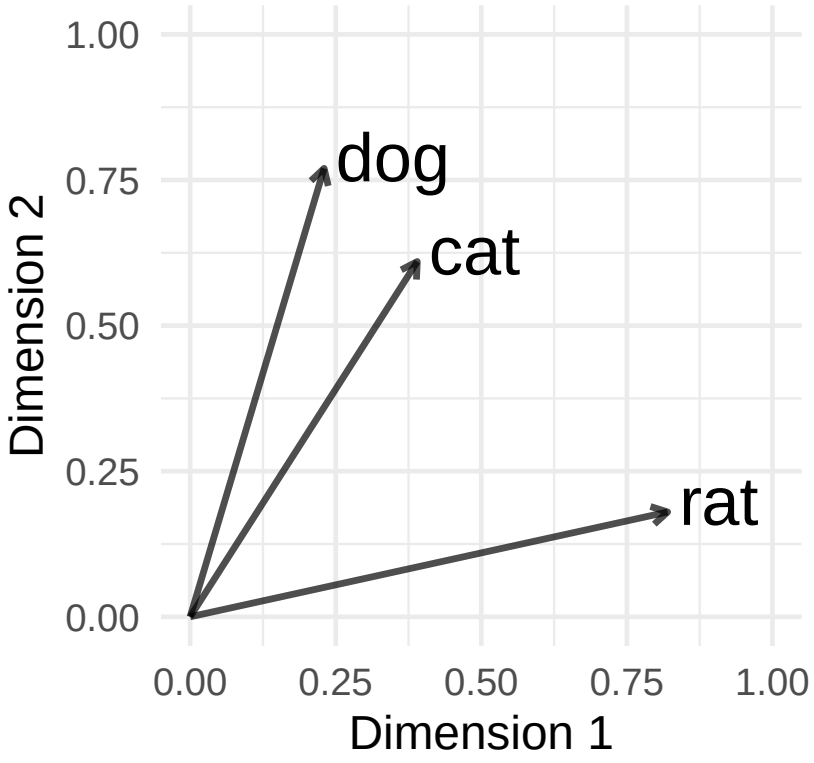
Then, you might have

- ➜ The word `cat` represented by `(0.39,0.61)`
- ➜ The word `dog` represented by `(0.23,0.77)`
- ➜ The word `rat` represented by `(0.82,0.18)`

Word embeddings have to be *estimated* from a corpus

- ➜ The key idea: words that appear in *similar contexts* get *similar vectors*
- ➜ Classic methods: **word2vec**, **GloVe** (pre-trained versions are downloadable)

Word embeddings



Machine "learns" these dimensions from patterns in the corpus

Analyst interprets their meaning

You can measure the similarity of words using **cosine similarity** (recall Lecture 9)

- ➜ Roughly: vectors pointing in the same direction are very "similar"
- ➜ One-hot encodings are all perfectly dissimilar using this metric

Estimating embeddings: a tiny example

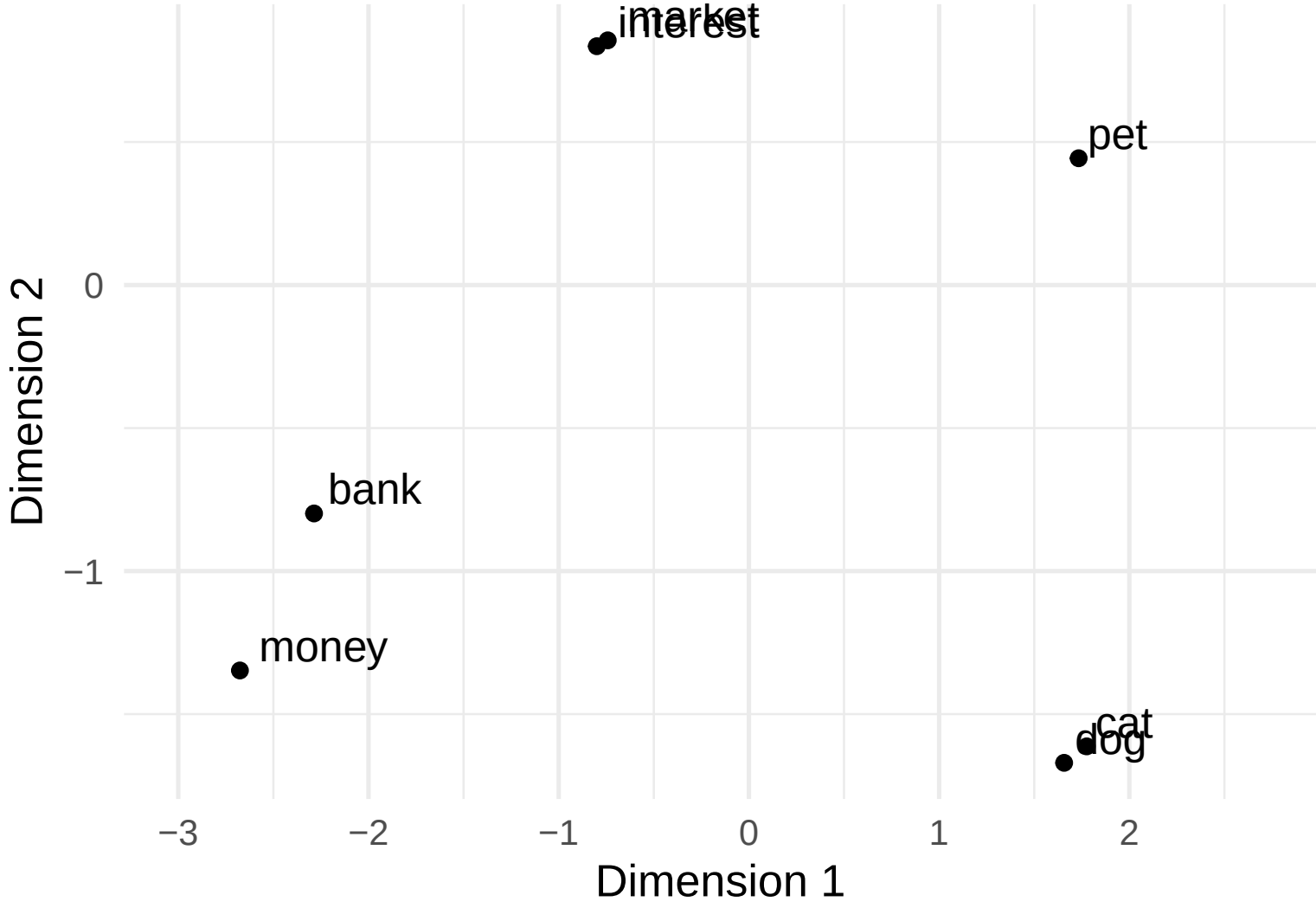
We can estimate embeddings ourselves: count which words appear *near* which, then reduce dimensions (PCA thinking, from Lecture 9!)

```
docs <- c("the dog chased the cat", "my dog is a loyal pet",
          "the cat is a lazy pet", "a cat and a dog make good pets",
          "deposit money in the bank", "the bank raised interest rates",
          "she withdrew money from the bank",
          "invest money in the stock market")
toks <- tokens(docs, remove_punct = TRUE)
X <- as.matrix(fcm(toks, context = "window", window = 3))  # co-occurrence
emb <- prcomp(X + t(X))   # PCA on the co-occurrence matrix
```

Each word's embedding: its first few **principal components**.

- ➜ Exactly the dimension reduction idea from Lecture 9
- ➜ word2vec and GloVe do (roughly) this at scale, more cleverly

Estimating embeddings: a tiny example



Dimension 1 separates *pets* from *finance*. Nobody told the machine those concepts exist: it found them in the co-occurrence patterns.

Measuring similarity between words

Recall **cosine similarity** from Lecture 9: vectors pointing the same way score near 1, orthogonal vectors score 0.

Each word's row of the co-occurrence matrix is its **context vector**: which words it appears near, and how often.

In our tiny corpus:

$$\text{cosine}(\text{dog}, \text{cat}) \approx 0.63 \qquad \text{cosine}(\text{dog}, \text{bank}) \approx 0.32$$

- ➜ `dog` and `cat` appear near the same words, so their vectors are similar
- ➜ `dog` and `bank` do not
- ➜ One-hot encodings would score 0 for *every* distinct pair

Why embeddings?

Use embeddings, not raw words, as features for text analysis. Why?

1. **Similarity is built in**: "dog", "puppy", "canine" get similar vectors
   - ➜ One-hot: "dog" and "puppy" are as unrelated as "dog" and "refrigerator"
2. **Denser and smaller**: far fewer dimensions than a sparse DFM
   - ➜ Models generalise better from denser features
3. **Geometry captures analogies**: king $-$ man $+$ woman $\approx$ queen

LLMs are built on the same idea: step one is to embed every token

One limitation: each word gets *one fixed vector* wherever it appears

➜ "bank" the river and "bank" the money: same vector.

Word embeddings

Analogy: king − man + woman = queen